

Agent 自主性分级

分级模型

L0 规则自动化 → L1 Workflow → L2 半自主 Agent → L3 全自主 Agent。

四级定义

等级	名称	一句话定义	决策权	工具权限
L0	规则自动化	无 LLM 决策，代码/规则决定执行路径	无	固定函数，按代码执行
L1	Workflow	LLM 在固定图结构的节点内做生成、分类或判断	节点局部	图中预定义节点
L2	半自主 Agent	LLM 动态选择工具和步骤，但高危动作需人工确认或受策略限制	受边界约束的自主	白名单 + 分级审批
L3	全自主 Agent	在明确目标和护栏内，端到端自主规划、执行、验证与交付	高	受沙箱、预算与审计约束

每个等级细看

L0：规则自动化

没有大模型参与决策，或 LLM 只是无反馈的文本处理器。

- 典型例子：正则路由的客服分流、定时报表、规则化审核。
- 优点：可预测、可审计、成本极低。
- 局限：无法应对开放任务。

L1：Workflow

流程骨架固定，LLM 是图里的节点。控制权在 Node/Edge/State，不在模型。

- 典型例子：文章审核流水线——生成初稿 → 质量评分 → 不通过则修改，最多 3 轮。
- 关键设计：节点职责单一、条件边显式化、State 只放必要数据、循环配终止条件。
- 适用判断：执行路径能提前确定，或拆解后能确定。

L2：半自主 Agent

模型在 Agent Loop 内动态决策“下一步调用哪个工具”，但权限与动作空间被严格限制。

- 典型例子：故障排查 Agent——可以查监控、查日志、查知识库，但“发通知”“重启服务”“改配置”需要人工确认。
- 关键设计：
 - 工具分风险等级：只读 / 可逆写 / 不可逆写；
 - 高危工具二次确认；
 - 预算与轮次上限；
 - 所有工具调用留痕。
- 适用判断：路径不确定但动作风险可控，这是生产环境最推荐的上限。

L3：全自主 Agent

在长时间任务里自主规划、执行、验证，甚至自己修复失败。仅适合高收益且可控的封闭环境。

- 典型例子：代码库内自动修简单 Bug 并跑完整回归测试；数据中心无人值守巡检与自动修复。

- 前置条件：
- 强护栏：沙箱、权限最小化、预算硬上限；
- 强评测：任务成功率可量化，回归可自动验证；
- 强可观测：完整 Trace 和回放；
- 可回退：所有动作可撤销或环境可重建。

升级判断表

你面对的情况	建议等级
需求明确、流程稳定	L0 / L1
流程有固定骨架，但某些节点需要模型判断	L1
路径不确定，动作只读或低风险	L2
路径不确定，包含写操作但可审批	L2
封闭环境、动作可回滚、有强评测	L3（谨慎试点）
开放环境、不可逆动作、无法回滚	不允许 L3，回到 L1/L2

自主性不只是一个标签

同一系统内部不同环节可以有不同的自主性：

环节	推荐自主级
信息检索、日志查询	L2全自主
生成初稿、代码建议	L2 全自主
发消息、写工单	L2 半自主（确认后执行）
重启服务、数据库变更	L2 人工审批
权限变更、批量删除	禁止 Agent 直接执行

因此架构文档中不应只说“这是一个 Agent”，而要写清楚：**哪些工具可自主调用，哪些必须审批，哪些禁止调用。**

落地检查清单

- 是否给系统标注了 L0-L3 等级？
- 是否按风险等级对工具分类？
- 是否配置了轮次上限、Token 预算和超时？
- 高危动作是否有人工确认或自动回滚？
- 是否记录“谁、在何时、调用什么工具、参数是什么、结果如何”？
- 是否在 L3 前具备评测集和回归机制？
- 降级路径是否清晰：L3 失败能否降为 L2/L1？

相关文档

- 概念边界见 [Agent 术语表与概念边界](#)
- 工具权限落地见 [AI 应用系统设计](#)
- Agent Loop 与范式见 [AI Agent 核心概念](#)

记忆口诀：**L0 不决策，L1 走图，L2 自主但有边界，L3 全自主必须强护栏。**